Tipos de datos primitivos en Java

Nombre	Información representada	Rango	Descripción	Valor por defecto
byte	Números enteros	-128 .. +127	8 bits de longitud	0
short	Números enteros	-32768 .. +32767	16 bits de longitud	0
int	Números enteros	-2147483648 .. +2147483647	32 bits de longitud. Es el tipo entero que se usa por defecto[1]	0
long	Números enteros	-9223372036854775808 .. +9223372036854775807	64 bits de longitud. Para enteros muy grandes	0l / 0L
char	Un carácter (letra, dígito, símbolo…)	Cualquier carácter Unicode	Guarda un solo carácter, entre comillas simples: `'A'`	'\u0000'
float	Números decimales	Precisión → Unas 7 cifras exactas (como una calculadora con pantalla pequeña)	Coma flotante de 32 bits. Se escribe con `f/F` al final Ejemplo: `float f = 3.14f;`	`0.0f / 0.0F`
double	Números decimales	Precisión → Unas 15-16 cifras exactas (calculadora con pantalla más grande, más exacto que float)	Coma flotante de 64 bits. Es el tipo decimal que se usa por defecto[1] Ejemplo: `double d = 3.14;`	`0.0d / 0.0D`
boolean	Verdadero o falso	true / false	Se usa para condiciones y comprobaciones	false

(1)
Cuando escribes un número en tu código sin indicar explícitamente su tipo, Java decide por ti qué tipo de dato es:

- Si el número es entero (sin coma), como **100**, Java lo trata automáticamente como **int.**
- Si el número tiene decimales, como **3.14**, Java lo trata automáticamente como **double**.

Por eso decimos que int es el tipo entero "por defecto" y double el tipo decimal "por defecto": son los que Java asume si no le dices lo contrario.
Esto es importante porque, si quieres usar otro tipo, tienes que avisar a Java con un sufijo.
Por ejemplo, si declaras

float pi = 3.14;

dará error, porque 3.14 ya ha nacido como double antes de que puedas guardarlo en la variable, y Java no permite guardar un double en un float sin más.
La solución es escribir

float pi = 3.14f;

añadiendo la f al final para decirle a Java "quiero que este número sea float, no double".

Una cosa importante sobre float y double

Los ordenadores guardan los números decimales por dentro en **binario**, no en el sistema decimal que usamos nosotros. Igual que `1/3` no se puede escribir de forma exacta en decimal (`0,3333...`, interminable), algunos números decimales normales y corrientes tampoco se pueden escribir de forma exacta en binario. Esto provoca, a veces, errores de redondeo diminutos e invisibles.

No hace falta entender el porqué todavía — solo quédate con esta regla práctica: nunca compares dos números `float` o `double` con == **para ver si son iguales.** En vez de preguntar si son iguales, pregunta si están muy muy cerca.